

Виртуальная CAN-плата

Драйвер символического устройства для эмуляции температурного контроля.

Андрей Юнин • разработка ядра Linux • уровень Medium

Воспроизводимый источник CAN-сообщений о состоянии платы.

Для проверки управляющего ПО без физического оборудования.
Температуру задаём из терминала; состояние и события наблюдаем в Linux.

Результат проекта: модуль ядра, утилита `iocli` и воспроизводимые проверки.

Назначение и объём — из согласованного описания проекта.

Зачем эмулировать температурную плату

Разработчику управляющего ПО нужны повторяемые нормальные и аварийные ситуации.

Контекст проекта: проверка ПО электронных плат на обычном Linux ПК.

Что нужно проверить

Нормальную работу
Перегрев и низкую
температуру
Возврат в допустимый
диапазон

На физической плате

Нужен испытательный
стенд.
Подготовка температурных
условий требует времени.

В нашем проекте

Температура задаётся
командой.
Модуль вычисляет статус.
Сообщения получает
программа или sandump.

Пользователь задаёт условия; виртуальная плата выдаёт соответствующие сообщения.

Температура — эмулируемое показание датчика; физического измерения нет.

В основе — символьное устройство

Модель платы реализована в драйвере; приложения обращаются к ней через `/dev/virtual_board`.

Прикладная задача: CAN-плата • механизм Linux: драйвер символьного устройства

Доступ через `/dev`

`write` — команды платы
`read` — события процесса
`ioctl` — параметры и состояние

Логика в драйвере

Общая температура и пределы
Определение статуса
Запуск и остановка CAN
Хранение событий

Особенность `Medium`

Два процесса открывают устройство.
У каждого — свой FIFO.
Каждый получает свою копию события.

Одно состояние платы, три операции устройства и отдельная очередь событий каждого процесса.

Обязательные операции: `read` / `write` / `ioctl` • особенность `Medium`: персональные буферы.

Одна команда — три видимых результата

Плата уже запущена; пределы 10–70 °C; читатель и candump открыты заранее.

write → board.status → локальное событие + CAN-уведомление

```
printf 'temp 850\n' | sudo tee /dev/virtual_board
```



Снимок / boardctl status

```
temperature_decic = 850
temp_status = overheat
state = running
```



Событие / cat /dev/...

```
event=temp_transition
temp=850
temp_status=overheat
(остальные поля сокращены)
```



CAN / candump vcan0

```
ID: 124   DLC: 8
52 03 64 00 BC 02 02 01
```

Периодические 123 продолжаются

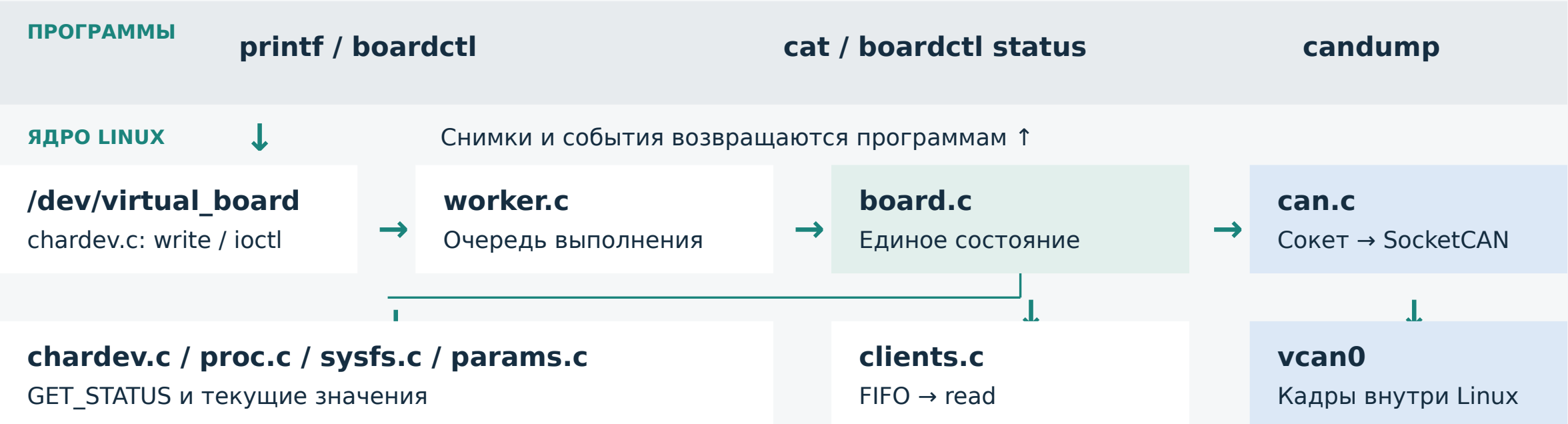
temp 850 = 85,0 °C: плата обнаруживает перегрев, но продолжает передачу.

Иллюстрация поведения по протоколу; это не снимок одного сохранённого запуска.

Где находится виртуальная плата

Пользовательские программы обращаются к модулю через интерфейсы Linux.

tools/boardctl.c → src/ → SocketCAN / пользовательские чтения



Состояние платы одно; снимки, события и CAN-кадры — разные способы его наблюдать.

Наш код: прямоугольники с именами файлов. SocketCAN и vcan — готовые механизмы Linux.

Файлы повторяют обязанности модуля

Цветная строка внизу следующих слайдов отмечает файлы текущего объяснения.

Kbuild объединяет `src/*.o` в `virtual_board.ko`; `board.h` связывает внутренние объекты

main.c + params.c

Загрузка, завершение, параметры

chardev.c

Регистрация `/dev`; `write` и `ioctl`

worker.c

Workqueue, completion, hrtimer

board.c

Температура, пределы и состояния

clients.c

Процессы, список, FIFO и `read`

can.c

Сокет, кадр, отправка, ошибки

proc.c + sysfs.c

Сводка и отдельные атрибуты

board.h

Общие внутренние объекты

tools/boardctl.c + UAPI

Утилита и общий контракт `ioctl`

Общий контекст: `board.h`. Граница с приложением: `include/virtual_board_uapi.h`.

Внутренние структуры и пользовательский ABI отделены друг от друга.

Makefile: собрать, загрузить, выгрузить

Модуль собирается через систему сборки ядра; Kbuild задаёт состав его объектов.

Makefile / Kbuild → virtual_board.ko; tools/boardctl.c → пользовательская утилита

MAKEFILE / Makefile / ФРАГМЕНТ ТЕКУЩЕГО ФАЙЛА

```
all: make boardctl
load: make
    sudo insmod ./virtual_board.ko
unload:
    sudo rmmod virtual_board
make:
    $(MAKE) -C $(KDIR) M=$(CURDIR) modules
```

make

Создаёт virtual_board.ko и tools/boardctl.
Использует заголовки работающего ядра.

make load

Собирает модуль и выполняет insmod.
Перед этим нужен интерфейс vcan0.

make unload

Выполняет rmmod. Сначала stop и закрытие всех читателей устройства.

Цели load и unload вызывают insmod и rmmod; это часть Makefile проекта.

До загрузки: заголовки текущего ядра и готовый vcan0. Полные команды — в README.

Три разных запроса к плате

Управление и наблюдение используют одно состояние `board.status`.

`chardev.c / clients.c / proc.c / sysfs.c ↔ tools/boardctl.c`

Изменить

write: temp, limits, start, stop
ioctl: SET_LIMITS, SET_PERIOD

Узнать сейчас

ioctl GET_STATUS: вся плата
sysfs: отдельное значение
procfs: плата и клиенты

Получить событие

read: событие из FIFO
процесса
CAN: кадры для внешнего
наблюдателя

ioctl — операция того же `/dev/virtual_board`; `boardctl` вызывает её из `userspace`.

API: `file_operations` • `unlocked_ioctl` • `proc_ops` • `DEVICE_ATTR_RO`

Почему один cat ждёт, а другой отвечает

Событие хранит прошлое изменение; снимок показывает текущее значение.

clients.c: vb_client_read • chardev.c: vb_ioctl • sysfs.c: state_show

cat /dev/virtual_board

Открывает пустой FIFO.
Ждёт новое событие.
После строки снова читает.

boardctl status

Вызывает GET_STATUS.
Получает согласованный
снимок.
Завершается.

cat .../state

Вызывает sysfs show.
Получает running или
stopped.
Завершается.

Такое поведение задано нашими обработчиками.

API: wait_event_interruptible • copy_to_user • sysfs_emit

write: от команды к изменению платы

850 означает 85,0 °C; верхний предел в этом примере — 70,0 °C.

chardev.c → worker.c → board.c → clients.c / can.c

01 / Вход

write → копирование строки
Проверка команды
Собственная копия в ядре

02 / Выполнение

Заявка → ordered workqueue
worker → изменение
состояния
normal → overheat

03 / Выход

Событие → FIFO → read
Снимок → ioctl / sysfs / proc
При running → CAN 0x124

Изменение внутри прежнего температурного статуса не создаёт новый переход.

API: copy_from_user • queue_work • mutex_lock • kfifo_in • kernel_sendmsg

ioctl: управление и снимок состояния

Утилита boardctl открывает /dev/virtual_board и вызывает ioctl с общим UAPI.

tools/boardctl.c → chardev.c: vb_ioctl; include/virtual_board_uapi.h — общий контракт

С / src/chardev.c / ФРАГМЕНТ ТЕКУЩЕГО ФАЙЛА

```
case VB_IOC_SET_PERIOD:
    // ... проверка права записи
    if (copy_from_user(&period, user_arg,
                      sizeof(period)))
        return -EFAULT;
    command.type = VB_CMD_SET_PERIOD;
    command.period_ms = period.period_ms;
    break;
// ... общий путь изменяющих команд:
return vb_submit_command(&command);
```

SET LIMITS / SET PERIOD

boardctl передаёт структуру. Драйвер копирует аргумент и ставит команду в общую очередь.

GET STATUS

Согласованный снимок под state_lock; copy_to_user возвращает его приложению.

Наблюдаемый результат

limits / period меняют параметры. status и sysfs отражают сохранённые значения.

SET_LIMITS / SET_PERIOD изменяют плату через worker; GET_STATUS читает снимок напрямую.

API: _IOW / _IOR • copy_from_user / copy_to_user • unlocked_ioctl

Температура и передача независимы

В board.status хранятся два разных признака.

board.c: vb_calc_temp_status / vb_board_apply_command

state

stopped / running
Разрешает периодическую передачу и CAN-уведомления.

temp_status

undertemp / normal / overheat
Вычисляется по температуре и текущим пределам.

Наблюдаемый пример

start → running / normal
temp 850 → running / overheat
stop → stopped / overheat

stop сохраняет температуру и её диагноз.

Инструменты: enum • единая структура состояния • state_lock

Как приложение находит наш драйвер

При загрузке связываем номер устройства, объект cdev и таблицу файловых операций.

chardev.c: vb_chardev_init → board.devno / board.cdev / vb_fops

1 / Номер устройства

alloc_chrdev_region

Выделяет major/minor.

Номер сохраняется в devno.

2 / Операции драйвера

cdev_init → vb_fops

cdev_add → devno

Связь номера с
обработчиками
зарегистрирована.

3 / Доступ приложению

class_create / device_create

Появляются объект и
узел /dev.

open → файловые операции
нашего драйвера.

/dev/virtual_board содержит тип и номер устройства; состояние и события хранит драйвер.

API: alloc_chrdev_region • cdev_init / cdev_add • class_create / device_create

Регистрация интерфейсов: /dev, /proc, /sys

После успешной загрузки доступны /dev, /proc и /sys; при выгрузке интерфейсы удаляются.

chardev.c: vb_chardev_init • sysfs.c: vb_sysfs_init • proc.c: vb_proc_init

/dev	/dev/virtual_board	cdev_add + device_create file_operations: open / read / write / ioctl
/proc	/proc/virtual_board	proc_create + proc_ops Сводка платы и клиентов; снимок при open
/sys	/sys/class/virtual_board/virtual_board/	class_create / device_create device_create_file: state и temp_status

В init каждый интерфейс получает свои обработчики; в exit его регистрация удаляется.

Дополнительно: /sys/module/virtual_board/parameters/ — числовые параметры с правами 0444.

read / write / ioctl: таблица обработчиков

read, write и ioctl символьного устройства связаны с обработчиками через vb_fops.

chardev.c: vb_fops — адреса функций связывают операции с нашим кодом

С / src/chardev.c / ФРАГМЕНТ ТЕКУЩЕГО ФАЙЛА

```
static const struct file_operations vb_fops = {  
    .owner = THIS_MODULE,  
    .open = vb_client_open,  
    .release = vb_client_release,  
    .read = vb_client_read,  
    .write = vb_write,  
    .unlocked_ioctl = vb_ioctl,  
};
```

Кто вызывает

Процесс делает системный вызов. VFS находит операцию устройства и вызывает нашу функцию.

Что связывает

cdev_init связывает cdev с vb_fops. Имена функций справа — наш код.

Где живёт логика

write/ioctl — chardev.c; open/read/release — clients.c.

Символьное устройство передаёт операции приложения нашим функциям драйвера.

API ядра: struct file_operations, cdev_init • наши функции: vb_*

Команда ждёт результата worker

Одна заявка связывает данные, работу, результат и уведомление.

worker.c: vb_submit_command и vb_command_work — фрагменты, пропуски отмечены

C / src/worker.c / ФРАГМЕНТ ТЕКУЩЕГО ФАЙЛА

```
request->command = *command;
init_completion(&request->done);
INIT_WORK(&request->work, vb_command_work);
queue_work(board.wq, &request->work);
wait_for_completion(&request->done);
ret = request->result;
// ... vb_command_work:
request->result =
    vb_board_apply_command(&request->command);
complete(&request->done);
```

Передать

В заявке уже лежит копия команды. queue_work поручает её выполнение worker.

Дождаться

wait_for_completion приостанавливает отправителя. Worker записывает result до complete.

Освободить

После уведомления отправитель читает результат и освобождает заявку.

После complete обработчик больше не обращается к заявке.

API: kmalloc • INIT_WORK • wait_for_completion • complete • kfree

От start до периодического CAN-кадра

Таймер назначает работу; отправка выполняется рабочим потоком.

board.c: start → worker.c: callback / periodic_work → can.c: send_status

01 / Запуск

Команда start
state = running
hrtimer_start(interval)

02 / Срок наступил

vb_timer_callback
queue_work(tx_work)
hrtimer_forward_now
HRTIMER_RESTART

03 / Отправка

vb_periodic_work
Снимок под state_lock
Проверка running
CAN-кадр 0x123

Первый периодический кадр ожидается через интервал после start.

API: hrtimer • CLOCK_MONOTONIC • ordered workqueue • kernel_sendmsg

Четыре строки связывают таймер и worker

Callback выполняется без ожидания, mutex и отправки через сокет.

worker.c: vb_timer_callback — тело без комментариев

C / src/worker.c / ФРАГМЕНТ ТЕКУЩЕГО ФАЙЛА

```
ctx = container_of(timer, struct board_ctx, tx_timer);
queue_work(ctx->wq, &ctx->tx_work);
hrtimer_forward_now(timer, ctx->tx_interval);
return HRTIMER_RESTART;
```

Найти объект

Ядро передало адрес timer. container_of находит содержащий его board_ctx.

Назначить работу

Callback ставит tx_work, переносит срок и возвращается. Отправит кадр worker.

Остановить

stop отменяет таймер; уже поставленная работа проверяет stopped и пропускает отправку.

Регистрация callback, назначение срока и выполнение работы — три разных действия.

API: container_of • queue_work • hrtimer_forward_now • HRTIMER_RESTART

Как кадр добирается до candump

CAN передаёт короткие сообщения с ID. Здесь обмен идёт внутри Linux через vcan0.

worker.c / board.c → can.c → SocketCAN → vcan0 → сокет candump

Отправитель в ядре

Среда Linux

Получатель

can.c

Снимок → struct can_frame
Сокет PF_CAN / CAN_RAW



SocketCAN + vcan0

Обработка отправки
Локальная доставка кадра



Сокет candump

Отдельный CAN_RAW-сокет
Читает сообщения vcan0

```
candump vcan0
vcan0 123 [8] FA 00 64 00 BC 02 00 01
vcan0 124 [8] 52 03 64 00 BC 02 02 01
```

Иллюстрация формата вывода: статус 25 °C и переход к перегреву 85 °C.

В vcan нет проводов и контроллера: проверяем сообщения и логику программы.

SocketCAN — подсистема CAN в Linux. vcan0 — программный интерфейс. candump — получатель.

Имя `vsap0` превращается в привязку сокета

Интерфейс готовит пользователь. `can.c` находит его и создаёт собственный `CAN_RAW`-сокет.

`can.c`: `vb_can_get_ifindex` / `vb_can_init` — фрагменты, проверки между ними опущены

C / `src/can.c` / ФРАГМЕНТ ТЕКУЩЕГО ФАЙЛА

```
dev = dev_get_by_name(&init_net, can_iface);
// ... проверка dev и ARPHRD_CAN
ret = dev->ifindex;
dev_put(dev);
// ... результат поиска -> addr.can_ifindex
addr.can_ifindex = ret;
ret = sock_create_kern(&init_net, PF_CAN,
    SOCK_RAW, CAN_RAW, &board.can_sock);
// ... проверка ret, пустой CAN_RAW_FILTER
ret = kernel_bind(board.can_sock,
    (struct sockaddr *)&addr, sizeof(addr));
```

1 / Найти

По имени получаем `net_device`; проверяем `CAN`-тип и сохраняем `ifindex`.

2 / Создать

`PF_CAN` + `SOCK_RAW` + `CAN_RAW`. Сокет хранится в общем контексте платы.

3 / Привязать

`kernel_bind` выбирает интерфейс для отправок. Приём модулю не нужен.

`board.can_sock` хранит адрес сокета; `sockaddr_can` задаёт интерфейс через `ifindex`.

API: `dev_get_by_name` • `dev_put` • `sock_create_kern` • `kernel_bind`

85 °C превращаются в байты CAN-кадра

CAN ID отвечает «какое сообщение», data[8] — «какие значения».

can.c: vb_can_fill_frame — байты data[] и поля struct can_frame

can_id = 0x124
Уведомление о переходе

len = 8
Длина полезных данных

data[0..7]
Явное кодирование значений

байт 0	байт 1	байт 2	байт 3	байт 4	байт 5	байт 6	байт 7
52	03	64	00	BC	02	02	01
850 = 85,0 °C		100 = 10,0 °C		700 = 70,0 °C		overheat	running

```
put_unaligned_le16((u16)status->temperature_decic, &frame->data[0]);
```

0x123 — статус по таймеру; 0x124 — изменение температурного диагноза.

API: struct can_frame • put_unaligned_le16 • CAN_MAX_DLEN

Кадр передаётся сетевому стеку

Локальный frame уже заполнен. kvec описывает адрес и размер передаваемой структуры.

can.c: vb_can_send_frame — фрагменты, пропуски отмечены

C / src/can.c / ФРАГМЕНТ ТЕКУЩЕГО ФАЙЛА

```
struct can_frame frame;
// ...
struct kvec iov = {
    .iov_base = &frame,
    .iov_len = sizeof(frame),
};
// ...
vb_can_fill_frame(&frame, can_id, status);
ret = kernel_sendmsg(board.can_sock,
    &msg, &iov, 1, sizeof(frame));
```

frame

Содержит идентификатор, длину и 8 байт data. Память выделена на стеке.

iov

Описывает существующий frame. Само присваивание адреса ничего не отправляет.

kernel_sendmsg

Передаёт кадр через привязанный сокет; возвращает число байтов или ошибку.

В SocketCAN передаём sizeof(frame), а полезная нагрузка протокола занимает 8 байт.

API: struct kvec • struct msghdr • MSG_DONTWAIT • kernel_sendmsg

CAN down: плата продолжает попытки

Сбой передачи не отменяет применённую команду и не меняет running.

can.c: vb_can_handle_result → clients.c: локальный CAN_ERROR

Интерфейс down

Первая ошибка →
CAN_ERROR
error = -100 / ENETDOWN
Повторы того же кода
подавлены

Интерфейс up

Следующие попытки
продолжаются
Успешная отправка
сбрасывает ошибку
Кадры возвращаются без
start

Повторный down

Новый сбой снова даёт
событие
Состояние остаётся running
stop завершает сценарий

Два down/up при одном start проверены; candump после down запускали заново.

Инструменты: единая обработка результата • отрицательный errno • локальные события

Каждое ожидание решает свою задачу

Общая очередь упорядочивает worker, но чтения интерфейсов идут параллельно.

board.h: command_lock / state_lock; board_client: fifo_lock / read_lock

Мьютексы

command_lock: отправители команд
state_lock: состояние и список
fifo_lock: FIFO и потери
read_lock: строка и позиция

Ожидание результата

completion в заявке
Отправитель ждёт
применения команды
Worker вызывает complete

Ожидание события

wait queue у клиента
Читатель ждёт доступные
данные
atomic_t хранит
pending_events

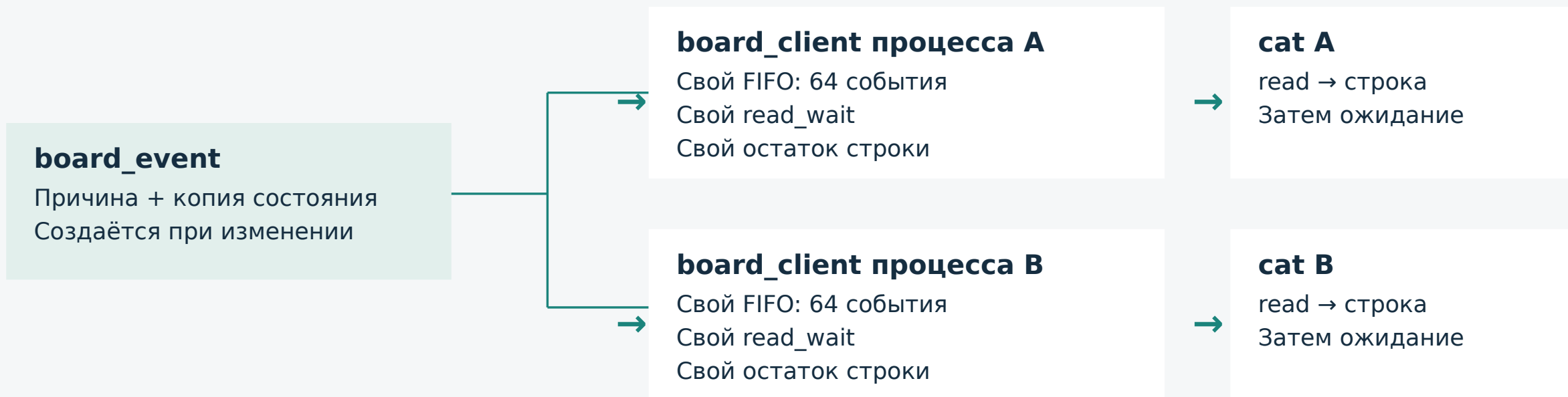
Таймер не берёт мьютексы. FIFO в текущем коде защищён mutex.

API: mutex • completion • wait_queue_head_t • atomic_t

Medium: персональный буфер процесса

У каждого процесса — свой FIFO; события получают процессы с открытием на чтение.

board.c: событие → clients.c: vb_clients_publish_locked



Чтение процесса A удаляет только его копию события. Копия в FIFO процесса B остаётся.

API: list_head • list_for_each_entry • DECLARE_KFIFO_PTR • kfifo_in

read: события из персонального FIFO

Ожидание не расходует процессор на постоянную проверку очереди.

clients.c: vb_client_read — фрагмент ветки пустого FIFO

C / src/clients.c / ФРАГМЕНТ ТЕКУЩЕГО ФАЙЛА

```
mutex_unlock(&client->read_lock);
if (file->f_flags & O_NONBLOCK)
    return -EAGAIN;
ret = wait_event_interruptible(
    client->read_wait,
    atomic_read(&client->pending_events) > 0);
if (ret)
    return ret;
```

Перед сном

Читатель отпускает read_lock, чтобы другие чтения могли работать.

Условие

pending_events учитывает FIFO и недочитанную строку. Публикация вызывает wake_up.

После пробуждения

Цикл повторно проверяет данные. copy_to_user выдаёт строку с сохранением остатка.

O_NONBLOCK возвращает EAGAIN; сигнал может прервать ожидание.

API: wait_event_interruptible • wake_up_interruptible • atomic_* • copy_to_user

Один процесс — один клиент и FIFO

Владелец определяется по TGID — идентификатору группы потоков процесса.

clients.c: vb_client_open / vb_client_release; board.h: struct board_client

Владелец — процесс

get_task_pid: PIDTYPE_TGID
Поиск клиента по
владельцу
При первом open — новый
клиент

Память и доступ

kzalloc: контекст клиента
kfifo_alloc: буфер событий
private_data: адрес клиента
opens: число открытий

Последнее закрытие

Удаление клиента из списка
kfifo_free: память FIFO
put_pid: ссылка владельца
kfree: контекст клиента

Повторные open и потоки одного процесса используют общий FIFO.

API: get_task_pid • put_pid • kzalloc • list_add / list_del • private_data

Одно состояние, разные формы чтения

Интерфейсы не хранят собственные копии температуры между запросами.

chardev.c: GET_STATUS • sysfs.c / params.c: атрибуты • proc.c: снимок

ioctl / GET_STATUS

Согласованная структура
24 байта, поля по 32 бита
Общий UAPI с boardctl
Порядок байтов хоста

sysfs

state и temp_status —
атрибуты
Числа — параметры модуля
Только чтение после
загрузки
Каждый файл читается
отдельно

procfs

Снимок при open
Плата, клиенты, lost_events
Чтение частями до EOF
Новый open обновляет
снимок

В проверке proc: clients 0 → 1 → 0; закрытие читателя сохранило running.

API: _IOR / _IOW • copy_to_user • sysfs_emit • proc_create • kzalloc

Ошибка ресурса: проверить и откатить

Проверяем результат до использования; при отказе освобождаем уже полученное.

clients.c: vb_client_alloc — фрагмент; аналогичные проверки есть в init подсистем

C / src/clients.c / ФРАГМЕНТ ТЕКУЩЕГО ФАЙЛА

```
client = kzalloc(sizeof(*client), GFP_KERNEL);
if (!client)
    return ERR_PTR(-ENOMEM);
ret = kfifo_alloc(&client->fifo,
    VB_EVENT_FIFO_CAPACITY, GFP_KERNEL);
if (ret) {
    kfree(client);
    return ERR_PTR(ret);
}
```

NULL

kzalloc не выделил контекст: возвращаем ошибку, поля client не используем.

IS_ERR / PTR_ERR

class_create и device_create возвращают ошибку в указателе. Для них нужна проверка IS_ERR.

Ошибка init

При отказе proc: удалить chardev/sysfs → завершить worker → освободить CAN-сокет.

Если FIFO не создан, контекст клиента освобождается; ошибка возвращается вызывающему.

API: kzalloc / kfree • kfifo_alloc • ERR_PTR / IS_ERR / PTR_ERR • отрицательный errno

exit: освобождение ресурсов по зависимостям

module_exit связывает выгрузку с vb_exit; он вызывает exit каждой подсистемы.

main.c: vb_exit — тело без журнала; далее module_exit(vb_exit)

C / src/main.c / ФРАГМЕНТ ТЕКУЩЕГО ФАЙЛА

```
vb_params_set_ready(false);
vb_proc_exit();
vb_chardev_exit();
vb_worker_exit();
vb_can_exit();
// ... регистрация обработчика выгрузки:
module_exit(vb_exit);
```

Интерфейсы

proc_remove и удаление sysfs-атрибутов; device/class/cdev и номер устройства освобождаются.

Отложенная работа

vb_worker_exit: hrtimer cancel, затем destroy_workqueue. Callback и работы завершены.

Сокет и клиенты

vb_can_exit вызывает sock_release. FIFO и контекст клиента освобождаются при последнем release.

Сначала прекращаем обращения и завершаем работу; затем освобождаем её ресурсы.

API: proc_remove • device_remove_file • cdev_del • hrtimer_cancel • sock_release

Что наблюдали в работающем модуле

Это результаты отдельных проверок; их не объединяем в один непрерывный запуск.

REPORT.md: реальные результаты отдельных проверок 21-22.09.2026

Температура и интерфейсы

temp 850 → overheat
Значения ioctl и sysfs согласованы
stop сохраняет диагноз
proc отражает появление клиента

События

Два независимых читателя
Одинаковая последовательность:
started → overheat → normal → stopped

Передача CAN

При period_ms=1000 измерен интервал 1,000026 с
После down/up кадры вернулись
Нового start не потребовалось

Источник результатов: отчёт проекта и выводы проверок 21-22 сентября 2026.

Средства наблюдения: boardctl • cat • candump • ip link

Что покажем при проверке проекта

Полные команды, исходные условия и ожидаемые результаты — в README репозитория.

README.md → «Инструкция для проверяющего»; в презентации — краткий сценарий.

Температура и CAN

25 → 85 → 50 → 5 °C

Норма → перегрев → норма
→ низкая температура
Границы 10 и 70 °C: normal

Два читателя и состояние

Независимые FIFO
Настройка через ioctl
Сверка sysfs и procfs
Отказ неверной команды

Остановка и завершение

stop прекращает CAN
Читатели закрываются
Выгрузка удаляет
интерфейсы
Down/up — дополнительный
тест

Два читателя видят свои события, candump — CAN, ioctl / sysfs / proc — состояние.

Открыть README: инструкция для проверяющего

Что демонстрирует этот проект

Пользовательские действия связаны с объектами и механизмами ядра.

Пользователь → интерфейсы → файлы и объекты → наблюдаемый результат

Архитектура

Единое состояние платы
Последовательное
изменение
Независимая доставка
событий
Управляемое время жизни

Учебные инструменты

cdev, procfs, sysfs и UAPI
hrtimer, workqueue, callbacks
mutex, completion, wait
queue
list, kfifo, PID API, память

Границы доказательств

vsan не проверяет
физическую CAN
Нет гарантии жёсткого real-
time
Расширенные испытания
неполны
Рабочая среда: x86_64

Интерфейсы объясняют поведение; схемы раскрывают, как ядро его обеспечивает.

Пробная редакция 3 • структура для первого знакомства с проектом